



Complete Learning Roadmap

From Zero to Enterprise Architect

Core Java • Frontend • Backend • Spring • Spring Boot • Microservices • DevOps

12+

Phases

200+

Topics

2-3

Years Journey

∞

Career Paths

■ Beginner

■ Intermediate

■ Advanced

■ Expert

■ TABLE OF CONTENTS

Phase 1	Core Java Fundamentals	1–3 months
Phase 2	Object-Oriented Programming (OOP)	1–2 months
Phase 3	Advanced Java	2–3 months
Phase 4	Java Build Tools & Version Control	2–3 weeks
Phase 5	Database & JDBC	1–2 months
Phase 6	Web & Frontend with Java	1–2 months
Phase 7	Spring Framework (Core)	1–2 months
Phase 8	Spring Boot	1–2 months
Phase 9	Spring Ecosystem (Security, Data, Cloud)	2–3 months
Phase 10	Microservices Architecture	2–3 months
Phase 11	DevOps, Cloud & Containers	2–3 months
Phase 12	Advanced Topics & Career Paths	Ongoing

■ BEGINNER PHASES

Build your foundation — the most critical step

Phase 1 Core Java Fundamentals

■ 1–3 months

Beginner

Environment Setup

- Install JDK (17+ LTS recommended)
- Set up JAVA_HOME & PATH
- IDE: IntelliJ IDEA / Eclipse / VS Code
- Hello World Program

Language Basics

- Data Types & Variables
- Operators & Expressions
- Control Flow (if/else, switch)
- Loops (for, while, do-while)
- Arrays (1D and 2D)
- String & StringBuilder
- User Input (Scanner)
- Type Casting & Conversion

Methods & Functions

- Method Declaration & Calling
- Parameters & Return Types
- Method Overloading
- Recursion
- Static vs Instance Methods
- Variable Scope & Lifetime

■ Practice on LeetCode easy problems | Write 50+ small programs | Learn to debug step-by-step

Phase 2 Object-Oriented Programming (OOP)

■ 1–2 months

Beginner

OOP Pillars

- Classes & Objects
- Constructors & this keyword
- Encapsulation & Access Modifiers
- Inheritance & super keyword
- Polymorphism (compile-time & runtime)
- Abstraction (abstract class)
- Interfaces & default methods

Advanced OOP

- Static members
- Final keyword
- Inner & Anonymous Classes
- Enum types
- instanceof operator
- Object class methods
- Wrapper Classes & Autoboxing

Design Principles

- SOLID Principles
- DRY & KISS
- Composition vs Inheritance
- UML Class Diagrams
- Design Patterns (Intro)

■ Build a Library Management System | Model real-world entities as classes

■ INTERMEDIATE PHASES

Go deeper — collections, concurrency, databases

Phase 3 Advanced Java

■ 2–3 months

Intermediate

Collections Framework

- List (ArrayList, LinkedList)
- Set (HashSet, TreeSet, LinkedHashSet)
- Map (HashMap, TreeMap, LinkedHashMap)
- Queue & Deque
- Iterator & ListIterator
- Collections utility class
- Comparable & Comparator

Generics & Functional

- Generic Classes & Methods
- Bounded Type Parameters
- Wildcards (? extends, ? super)
- Lambda Expressions
- Functional Interfaces
- Method References
- Stream API (map, filter, reduce)
- Optional class

Exception Handling

- try-catch-finally
- Checked vs Unchecked
- Custom Exceptions
- try-with-resources
- Multi-catch blocks

Java I/O & NIO

- File & FileReader/Writer
- BufferedReader/Writer
- Serialization
- NIO2 (Path, Files, Channels)
- Properties files

Concurrency

- Threads & Runnable
- Thread Lifecycle
- synchronized keyword
- wait/notify
- Executor Framework
- Callable & Future
- CompletableFuture
- Thread Safety & volatile
- Concurrent Collections
- AtomicInteger etc.

JVM Internals

- JVM Architecture
- Classloading
- Heap & Stack memory
- Garbage Collection algorithms
- JVM tuning flags
- Java Memory Model

■ Implement your own ArrayList | Write a multi-threaded bank system

Phase 4 Build Tools & Version Control

■ 2–3 weeks

Intermediate

Maven

- POM.xml structure
- Dependency management
- Build lifecycle phases
- Maven plugins
- Repository (local/central/remote)
- Multi-module projects

Gradle

- build.gradle (Groovy/Kotlin DSL)
- Tasks & dependencies
- Gradle wrapper
- Publishing artifacts

Git & GitHub

- git init, add, commit, push
- Branching strategies (Git Flow)
- Merge, Rebase, Cherry-pick

- Pull Requests & Code Review
- GitHub Actions (CI/CD intro)
- .gitignore

■ *Set up a Maven/Gradle project from scratch | Contribute to an open-source project*

Phase 5 Database & JDBC

■ 1–2 months

Intermediate

SQL Fundamentals

- DDL: CREATE, ALTER, DROP
- DML: SELECT, INSERT, UPDATE, DELETE
- JOINS (INNER, LEFT, RIGHT, FULL)
- Aggregations & GROUP BY
- Subqueries & CTEs
- Indexes & Performance
- Transactions & ACID

JDBC

- DriverManager & Connection
- Statement, PreparedStatement
- ResultSet navigation
- Connection Pooling (HikariCP)
- Batch operations
- Transaction management in JDBC

ORM / JPA

- JPA concepts & annotations
- Hibernate as JPA provider
- Entity mappings (@Entity, @Id, @Column)
- Relationships (@OneToMany etc.)
- JPQL & Criteria API
- Hibernate caching (L1/L2)
- N+1 problem & solutions

■ *Model a real schema (e-commerce) | Compare raw JDBC vs JPA performance*

Phase 6 Web & Frontend with Java

■ 1–2 months

Intermediate

Servlet & JSP (Legacy)

- HTTP request/response cycle
- Servlet lifecycle
- JSP & JSTL
- Session & Cookie management
- Filters & Listeners

Frontend Basics (for Java Devs)

- HTML5 & CSS3 fundamentals
- JavaScript ES6+ basics
- DOM manipulation
- Fetch API / AJAX
- JSON handling
- Thymeleaf template engine
- Bootstrap for quick UI

RESTful Web Services

- REST principles & HTTP verbs
- JAX-RS (Jersey)
- JSON & XML serialization
- API versioning strategies
- Postman for testing APIs
- Swagger / OpenAPI specification

■ Build a simple MVC web app without Spring | Create a REST API and test with Postman

■ ADVANCED PHASES

Industry-grade frameworks — Spring & Microservices

Phase 7 Spring Framework (Core)

■ 1–2 months

Advanced

Spring Core

- IoC Container & Dependency Injection
- Bean lifecycle & scopes
- @Component, @Service, @Repository, @Controller
- Constructor vs Setter vs Field injection
- ApplicationContext & BeanFactory
- @Configuration & @Bean
- Profiles & @Conditional

Spring AOP

- Aspect-Oriented Programming concepts
- Pointcuts & Joinpoints
- @Aspect, @Before, @After, @Around
- Transaction management via AOP
- Logging & Security cross-cutting concerns

Spring Data Access

- Spring JDBC Template
- Spring Data JPA
- Repository pattern
- Custom queries with @Query
- Pagination & Sorting
- Spring Transaction (@Transactional)

Spring MVC

- @Controller & @RestController
- @RequestMapping, @GetMapping etc.
- @RequestParam, @PathVariable, @RequestBody
- ModelAndView & Thymeleaf
- Exception handling (@ControllerAdvice)
- Validation (javax.validation / Hibernate Validator)
- File upload handling

■ Understand Spring without Boot first | Build a pure Spring MVC app with XML config

Phase 8 Spring Boot

■ 1–2 months

Advanced

Spring Boot Basics

- Spring Initializr setup
- Auto-configuration mechanism
- @SpringBootApplication
- application.properties / YAML
- Embedded Tomcat / Jetty / Undertow
- DevTools for hot reload

Building REST APIs

- RESTful controller design
- Request/Response DTOs
- MapStruct / ModelMapper
- Global Exception Handling
- Bean Validation with @Valid
- HATEOAS

Spring Boot Data

- Spring Data JPA with Boot
- H2 in-memory DB (dev)
- Flyway / Liquibase migrations
- Multiple datasources
- Spring Data MongoDB / Redis

Spring Boot Actuator

- Health & Info endpoints
- Metrics & Prometheus integration
- Custom actuator endpoints
- Environment & ConfigProps

Spring Boot Testing

- @SpringBootTest
- @WebMvcTest
- @DataJpaTest

- MockMvc & WebTestClient
- Testcontainers
- WireMock

■ *Build a full CRUD REST API with auth | Write 80%+ test coverage*

Phase 9

Spring Ecosystem (Security, Cloud, Messaging)

■ 2–3 months

Advanced

Spring Security

- Authentication & Authorization
- Basic & Form-based auth
- JWT (JSON Web Tokens)
- OAuth2 & OpenID Connect
- Method-level security (@PreAuthorize)
- CSRF & CORS config
- Custom UserDetailsService
- Keycloak integration

Spring Cloud (Intro)

- Service Discovery (Eureka)
- API Gateway (Spring Cloud Gateway)
- Config Server
- Circuit Breaker (Resilience4J)
- Distributed Tracing (Sleuth + Zipkin)

Messaging

- Spring Kafka (producer/consumer)
- Spring AMQP / RabbitMQ
- Event-driven architecture patterns
- Message serialization
- @KafkaListener, @RabbitListener

Caching & Performance

- @Cacheable, @CacheEvict
- Redis as cache store
- EhCache
- Response compression
- Async methods (@Async)

■ Secure your REST API with JWT + OAuth2 | Implement event-driven order processing

Phase 10	Microservices Architecture	■ 2–3 months	Advanced
Microservices Fundamentals			
• Monolith vs Microservices tradeoffs • Domain-Driven Design (DDD) • Bounded Contexts		• Service decomposition strategies • 12-Factor App methodology • API design for microservices	
Inter-Service Communication			
• Synchronous: REST (RestTemplate, WebClient) • Synchronous: gRPC • Asynchronous: Kafka / RabbitMQ		• Saga Pattern (orchestration vs choreography) • Service Mesh (Istio basics)	
Patterns & Resilience			
• Circuit Breaker (Resilience4J) • Bulkhead & Rate Limiting • Retry with exponential backoff • API Gateway pattern		• BFF (Backend for Frontend) • Strangler Fig pattern • CQRS & Event Sourcing	
Data Management			
• Database per service • Distributed transactions (2PC, Saga) • Event-driven data consistency		• Polyglot persistence • Shared database anti-pattern	
Observability			
• Centralized logging (ELK Stack) • Distributed tracing (Jaeger/Zipkin) • Metrics (Prometheus + Grafana)		• Health checks & Readiness probes • Alerting strategies	
■ Decompose a monolith step by step Build 3+ services communicating via Kafka			

■ EXPERT PHASES

Enterprise-grade — DevOps, Cloud, Architecture

Phase 11	DevOps, Cloud & Containers	■ 2–3 months	Expert
Docker			
• Dockerfile authoring • Multi-stage builds • Docker Compose		• Docker networking & volumes • Image optimization • Container registries (Docker Hub, ECR, GCR)	
Kubernetes			
• Pods, Deployments, Services • ConfigMaps & Secrets • Ingress controllers • Horizontal Pod Autoscaler		• Helm charts • Namespaces & RBAC • Persistent Volumes • kubectl commands	
CI/CD Pipelines			
• GitHub Actions workflows • Jenkins pipelines • GitLab CI/CD		• ArgoCD (GitOps) • Blue/Green & Canary deployments • Automated testing in pipelines	
Cloud Platforms			
• AWS: EC2, RDS, S3, EKS, Lambda • GCP: GKE, Cloud Run, BigQuery • Azure: AKS, App Service		• Infrastructure as Code (Terraform) • Serverless with AWS Lambda / Spring Cloud Function	
■ Containerize your Spring Boot app Deploy to K8s with zero-downtime rolling updates			

Phase 12	Advanced Topics & Architecture	■ Ongoing	Expert
Software Architecture			
• Hexagonal / Clean Architecture • Domain-Driven Design (advanced) • Event-Driven Architecture		• Reactive Programming (Project Reactor) • Reactive REST with WebFlux • Architecture Decision Records (ADRs)	
Performance Engineering			
• JVM profiling (VisualVM, JProfiler, async-profiler) • Heap dump analysis • GC tuning (G1, ZGC, Shenandoah)		• Load testing (JMeter, Gatling, k6) • Database query optimization • Connection pool tuning	
Advanced Testing			
• TDD & BDD (Cucumber) • Contract testing (Pact) • Performance testing		• Chaos Engineering • Mutation testing • Test Pyramid strategy	
Security Engineering			
• OWASP Top 10 for Java • SAST/DAST tools • Dependency vulnerability scanning		• Secrets management (Vault) • mTLS between microservices • Zero-trust architecture	
Career Specializations			

- Backend Engineer (REST APIs)
- Platform / Infrastructure Engineer
- Solutions Architect

- Tech Lead / Engineering Manager
- Open Source Contributor
- Java Champion path

■ *Read "Designing Data-Intensive Applications" | Contribute to Spring or Jakarta EE projects*

■ RECOMMENDED RESOURCES

■ Must-Read Books

- "Effective Java" — Joshua Bloch (bible for every Java dev)
- "Head First Java" — Sierra & Bates (visual, beginner-friendly)
- "Clean Code" — Robert C. Martin (writing readable, maintainable code)
- "Spring in Action" — Craig Walls (comprehensive Spring reference)
- "Java Concurrency in Practice" — Goetz et al.
- "Designing Data-Intensive Applications" — Martin Kleppmann (microservices/data)
- "Building Microservices" — Sam Newman
- "Domain-Driven Design" — Eric Evans (Blue Book)

■ Online Courses & Platforms

- Udemy: "Java Masterclass" — Tim Buchalka | "Spring Boot" — Chad Darby
- Coursera: Duke University Java Specialization
- Pluralsight: Java & Spring learning paths
- Baeldung.com — #1 Java & Spring tutorial website (bookmark this!)
- Spring.io/guides — Official Spring Framework guides
- JetBrains Academy — Hands-on Java track
- LeetCode / HackerRank — Algorithm practice in Java

■ Essential Tools & Technologies

- IDE: IntelliJ IDEA Ultimate (industry standard) / Eclipse / VS Code
- Build: Maven Central (search.maven.org) | Gradle build scans
- DB: PostgreSQL (prod), H2 (dev/test), Redis (cache), MongoDB (NoSQL)
- Messaging: Apache Kafka | RabbitMQ
- Containers: Docker Desktop | minikube (local K8s)
- Monitoring: Prometheus + Grafana | ELK Stack | Jaeger
- API Testing: Postman | HTTPie | curl
- Code Quality: SonarQube | Checkstyle | SpotBugs

■ COMPLETE TIMELINE OVERVIEW

Months	Phase	Focus	Milestone Project
1–3	Phase 1–2	Core Java + OOP	ATM Simulator / Bank System
4–6	Phase 3–4	Advanced Java + Tools	Multi-threaded Chat App
7–8	Phase 5–6	Database + Web	CRUD Web App with JDBC/JPA
9–10	Phase 7–8	Spring + Spring Boot	Full REST API + JWT Auth
11–13	Phase 9–10	Spring Ecosystem + Microservices	E-commerce Microservices
14–16	Phase 11	DevOps + Cloud	Deploy to AWS EKS via CI/CD
17+	Phase 12	Architecture + Specialization	Open Source Contribution

■ KEY PRINCIPLES FOR SUCCESS

- Build projects, not just tutorials
- Read open-source Java code (Spring, Guava)
- Understand the "why" before the "how"
- Document and blog what you learn
- Use real databases, not just in-memory
- Code every single day (even 30 min)
- Join Java communities (Reddit, Discord, Stack Overflow)
- Write tests for everything you build
- Never skip fundamentals — they always come back
- Deploy everything — dont just run locally